

Distillation Techniques

for the Bruise Segmentation Project

Nine knowledge-distillation techniques not yet used in this project, each mapped onto the actual SegFormer / YOLO26n-sem architecture, with a proposed diagram, loss formulation, and implementation hook.

Baseline already implemented:	response-based online KD, offline pseudo-label KD, cross-modal (ALS→WL) KD, fairness-reweighted KD
New in this document:	9 additional distillation families
Scope:	architecture diagrams, loss formulas, code hooks, priority recommendation

A technical proposal document, not a results report — no new numbers are claimed; every technique here is a candidate for future experimentation.

Contents

1	Executive Summary	2
2	Baseline: What This Project Already Does	3
3	Feature-Based (Hint) Distillation	4
3.1	What it is	4
3.2	Architecture for this project	4
3.3	Loss	4
4	Attention Transfer Distillation	6
4.1	What it is	6
4.2	Architecture for this project	6
4.3	Loss	6
5	Structured / Channel-Wise Distillation	8
5.1	What it is	8
5.2	Architecture for this project	8
5.3	Loss	8
6	Boundary-Aware Distillation	10
6.1	What it is	10
6.2	Architecture for this project	10
6.3	Loss	10
7	Multi-Teacher Ensemble Distillation (WL + ALS)	12
7.1	What it is	12
7.2	Architecture for this project	12
7.3	Loss	12
8	Deep Mutual Learning (Online Co-Distillation)	14
8.1	What it is	14
8.2	Architecture for this project	14
8.3	Loss	14
9	Self-Distillation (Deep Supervision)	16
9.1	What it is	16
9.2	Architecture for this project	16
9.3	Loss	16

10 Adversarial Fairness Distillation	18
10.1 What it is	18
10.2 Architecture for this project	18
10.3 Loss	18
11 Compression-Aware Distillation	20
11.1 What it is	20
11.2 Architecture for this project	20
11.3 Loss	20
12 Prioritized Recommendation	22
Appendix: Formula Reference	24

Chapter 1

Executive Summary

This document proposes nine distillation techniques that are *not* currently used anywhere in the bruise-segmentation pipeline, each adapted specifically to this project’s actual models (SegFormer-B2/B0, YOLO26n-sem) and data (paired ALS/WL photographs, ITA skin-tone groups) rather than presented generically. Every technique includes: what it matches between teacher and student, an architecture diagram, the loss term to add, which existing file it would hook into, and an honest note on cost/risk.

At a glance

Technique	Matches	Extra cost	Primary benefit
Feature (Hint)	intermediate encoder feature maps	small (adapters)	richer signal than logits alone
Attention Transfer	spatial attention maps	near-zero	cheap add-on, sharper focus
Structured/Channel-wise	per-channel spatial distributions	small	robust across capacity gap
Boundary-Aware	KD loss reweighted by edge distance	near-zero	sharper bruise boundaries
Multi-Teacher Ensemble	WL teacher + ALS teacher combined	moderate	one best-of-both WL student
Deep Mutual Learning	SegFormer $\leftrightarrow$ YOLO, live	moderate	no separate teacher training
Self-Distillation	network’s own deeper layers	small	improves the teacher itself
Adversarial Fairness	skin-tone signal in features	moderate	stronger equity than MMD alone
Compression-Aware	pruned/quantized vs. full model	small	real-time edge deployment

How to read this document

Chapters 2 gives the baseline for contrast. Chapters 3–11 are one technique each, ordered roughly from cheapest/simplest to most involved. Chapter 12 gives a prioritized recommendation given a limited GPU budget.

Chapter 2

Baseline: What This Project Already Does

Four distillation mechanisms already exist in this codebase; every technique in this document is an *addition to*, not a replacement of, these.

Mechanism	Where	What it matches
Online response-based KD (WL→WL)	pipeline/trainer.py, script 05	final-layer soft probabilities, live teacher forward pass
Offline pseudo-label KD (YOLO)	pipeline/mask_utils.py, script 08	fused pseudo-mask baked to disk once, used as if it were ground truth
Cross-modal KD (ALS→WL)	pipeline/trainer_als.py, script 16	teacher sees a <i>different photograph</i> (ALS) of the same bruise
Fairness-reweighted KD (Approach A/B)	pipeline/losses_fairness.py, script 12	final probabilities, reweighted per ITA skin-tone group; Approach B adds an MMD feature-alignment term

All four share one property worth naming explicitly: **every one of them matches only the final output probability** (Approach B is the sole exception, and only partially — it adds a global per-group feature-moment term on top of the same output-matching core). None of the four ever look at *how* the teacher’s intermediate representation reaches that answer, at the spatial structure of teacher/student agreement, or at bringing in more than one information source at once. That gap is exactly what the next nine chapters address.

Chapter 3

Feature-Based (Hint) Distillation

3.1 What it is

Response-based KD (this project’s current approach) only tells the student *what* the teacher’s final answer was. Feature-based distillation (FitNets-style hint learning) additionally tells the student *what the teacher was representing internally* at intermediate encoder stages, by matching feature maps rather than only the final probability map.

3.2 Architecture for this project

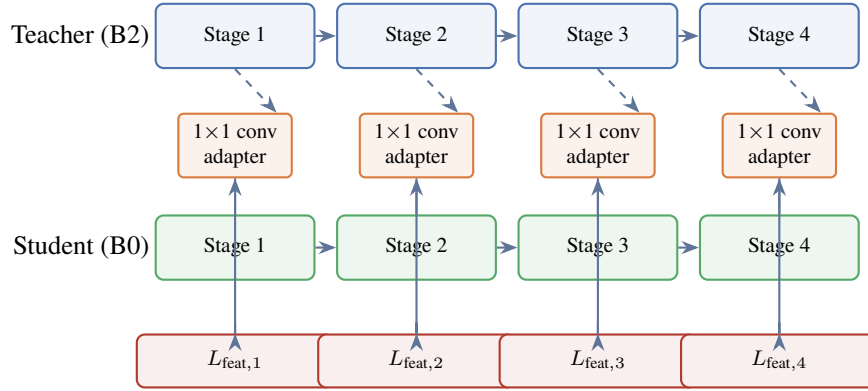


Figure 3.1: One 1×1 convolution "adapter" per encoder stage projects the student’s feature map to the teacher’s channel width; a stop-gradient copy of the teacher’s own stage output is the regression target. Four small per-stage losses are summed into the total feature loss.

3.3 Loss

$$L_{\text{feat}} = \sum_{i=1}^4 \|A_i(f_i^{\text{student}}) - \text{stopgrad}(f_i^{\text{teacher}})\|_2^2, \quad L = \alpha L_{\text{sup}} + (1 - \alpha) L_{\text{KD}} + \beta L_{\text{feat}}$$

where A_i is a learned 1×1 convolution (the only new trainable parameters this technique introduces) and β is a small weight (typically $\ll 1 - \alpha$, since the feature term is a secondary signal, not a replacement for the supervised or response-KD terms).

Implementation hook

HuggingFace's `SegformerModel` already returns per-stage hidden states when called with `output_hidden_states=True` — no architecture surgery is needed to *expose* the four feature maps, only to add the adapters and the loss term. This would live in `pipeline/losses.py` as a new `FeatureHintLoss` class, alongside the existing `DistillSegLoss`, and would need `pipeline/trainer.py`'s teacher-loading path to also return hidden states (currently `load_teacher()` returns a closure that only returns final logits).

Why it fits this project

SegFormer-B0 and SegFormer-B2 share the same 4-stage hierarchical structure and the same spatial resolution schedule ($H/4, H/8, H/16, H/32$) — only channel widths differ. That means every stage has a well-defined, same-shape counterpart to match, with no resolution mismatch to work around. This is one of the cleanest possible pairings for hint-style distillation.

Caveat

Feature magnitudes differ a lot between a 27M-parameter and a 3.7M-parameter network; without careful normalization (e.g. matching normalized/whitened features rather than raw activations) the adapters can end up just learning a scale factor rather than a meaningful representation match. Worth validating the intermediate feature loss is actually decreasing faster than a trivial scale-only fit would achieve, on a short trial run before committing a full training budget.

Chapter 4

Attention Transfer Distillation

4.1 What it is

A cheaper cousin of feature-hint distillation: instead of matching full feature maps (which needs a learned channel adapter), compress each feature map down to a single-channel *attention map* — essentially “where in the image is this stage paying attention” — and match only that. No learnable adapter is required at all.

4.2 Architecture for this project

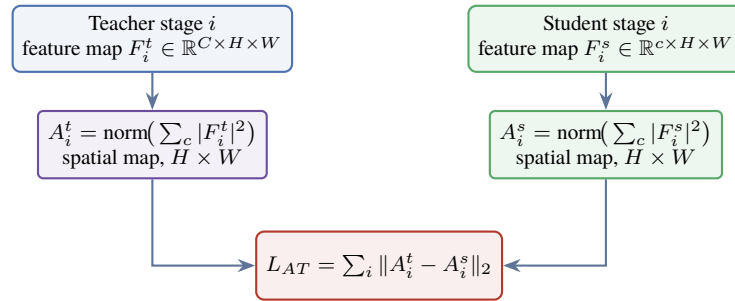


Figure 4.1: Both teacher and student compress their per-stage feature map to a single normalized spatial-attention map by summing squared activation across channels — collapsing the channel-count mismatch (256 vs. 32, say) away entirely, since the result is always $H \times W$ regardless of channel width. No adapter, no new trainable parameters.

4.3 Loss

$$A_i = \frac{\text{vec}(\sum_c |F_i[c, :, :]|^2)}{\|\text{vec}(\sum_c |F_i[c, :, :]|^2)\|_2}, \quad L_{AT} = \sum_{i=1}^4 \|A_i^{\text{teacher}} - A_i^{\text{student}}\|_2$$

Implementation hook

Zero new parameters, so this can be added as a pure function inside `pipeline/losses.py` with no change to either model’s architecture — the cheapest technique in this document to implement and to run. A natural first thing to try alongside feature-hint distillation, sharing the same hidden-state extraction plumbing.

Why it fits this project

Directly encourages the student to look at the *same spatial regions* the teacher does when forming its answer — for a task where the target (a bruise) is a spatially localized region against a mostly-irrelevant background, this is a more targeted signal than raw feature magnitude matching, and considerably cheaper than it.

Chapter 5

Structured / Channel-Wise Distillation

5.1 What it is

Plain feature or logit matching compares *raw values* pixel-by-pixel, independently. Structured distillation instead matches *relationships*: either the per-channel spatial *distribution* of activations (so a 27M-parameter teacher and a 3.7M-parameter student can be compared on equal footing regardless of raw activation scale), or a pixel-to-pixel *affinity matrix* capturing which regions the model considers similar to which other regions.

5.2 Architecture for this project

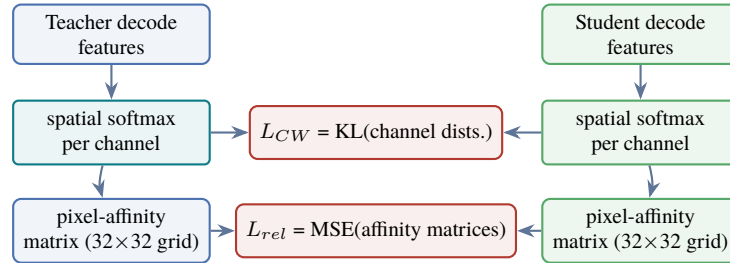


Figure 5.1: Two structured comparisons run in parallel: channel-wise (each channel’s spatial map treated as a probability distribution, compared via KL divergence – scale-invariant), and pairwise/relational (a subsampled pixel-affinity matrix capturing which regions the model treats as similar, compared via MSE).

5.3 Loss

$$L_{CW} = \sum_c \text{KL}(\sigma(F^t[c]/T) \parallel \sigma(F^s[c]/T)), \quad L_{rel} = \|\text{sim}(F^t) - \text{sim}(F^s)\|_2^2$$

where $\sigma(\cdot)$ is a softmax taken over the flattened spatial dimension of one channel, and $\text{sim}(F)$ is the cosine-similarity matrix between a fixed grid of subsampled pixel locations.

Implementation hook

Both terms operate on the decode-head feature tensor that `losses_fairness.py`’s Approach B already extracts for its MMD term — the pixel-affinity and channel-softmax computations would sit naturally alongside `FeatureAlignFairLoss` in the same file, reusing its feature-extraction hook rather than adding a new one.

Caveat

The 32×32 affinity matrix is a hyperparameter trade-off: too coarse and it loses the fine boundary detail this project already cares about (surface Dice); too fine and the $O(n^2)$ pairwise matrix becomes the dominant compute cost of the whole loss. Worth starting coarse and increasing only if the channel-wise term alone under-delivers.

Chapter 6

Boundary-Aware Distillation

6.1 What it is

The existing response-based KD loss treats every pixel equally: a pixel deep inside the bruise and a pixel exactly on its edge contribute the same weight to the loss. But this project’s own boundary metrics (Surface Dice, ASD, HD95, already computed in `pipeline/metrics_extended.py`) exist precisely because edge quality is not fully captured by plain Dice. This technique reweights the KD loss itself by distance to the ground-truth boundary, so the student is pushed hardest to match the teacher exactly where it matters most.

6.2 Architecture for this project

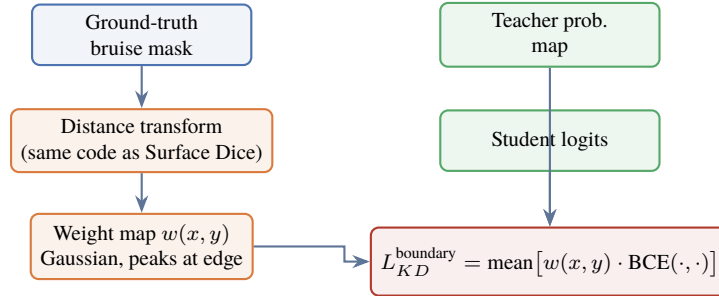


Figure 6.1: The same distance-transform machinery already used to compute Surface Dice/ASD/HD95 at evaluation time is reused at *training* time to build a per-pixel weight map, which multiplies the existing pixel-wise KD BCE term – structurally identical to how `losses_fairness.py` already reweights the KD term by ITA group, just keyed by boundary distance instead.

6.3 Loss

$$w(x, y) = \exp\left(-\frac{d(x, y)^2}{2\sigma^2}\right), \quad L_{KD}^{\text{boundary}} = \text{mean}_{(x, y)} \left[w(x, y) \cdot \text{BCE}(z_s(x, y), p_t(x, y)) \right]$$

where $d(x, y)$ is the signed distance from pixel (x, y) to the nearest ground-truth boundary pixel, and σ controls how narrow the boundary-focus band is.

Implementation hook

`pipeline/metrics_extended.py` already contains a distance-transform implementation used for Surface Dice/HD95 at evaluation time; the same function (or a thin training-time variant of it) produces $w(x, y)$. The weighted BCE itself is a one-line change to `DistillSegLoss` in `pipeline/losses.py`.

Why it fits this project

This is the single cheapest technique in this document to validate: it requires no new model parameters, no architecture change, and reuses code that already exists in the project for a different purpose. A natural very-first experiment.

Chapter 7

Multi-Teacher Ensemble Distillation (WL + ALS)

7.1 What it is

This project currently trains *two separate* students from two separate teachers: a WL-only student distilled from the WL teacher (script 05), and a WL-only student distilled from the ALS teacher (script 16). They are never combined. Multi-teacher ensemble distillation trains *one* WL-only student against a single soft target built from *both* teachers at once.

7.2 Architecture for this project

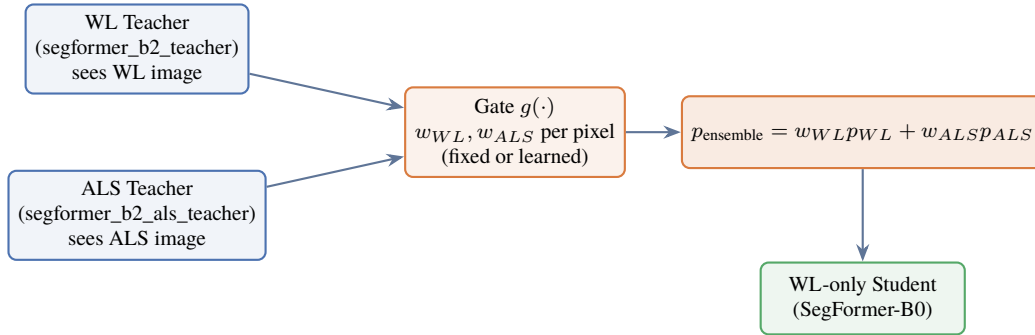


Figure 7.1: Both existing teachers run in parallel (WL teacher on the WL photo, ALS teacher on the paired ALS photo, same pairing already used by cross-modal KD); a gate combines their soft predictions into one ensemble target before a single WL-only student is trained against it.

7.3 Loss

$$p_{ensemble}(x, y) = g(x, y) \cdot p_{WL}(x, y) + (1 - g(x, y)) \cdot p_{ALS \rightarrow WL}(x, y)$$

The gate g can be as simple as a fixed scalar (e.g. $g = 0.5$), or per-image based on which teacher is more confident (higher peak probability), or – most ambitiously – a small learned network conditioned on ITA group, letting the ensemble lean on whichever teacher empirically performs better for that skin-tone group.

Implementation hook

Both teacher-loading paths already exist independently (`_load_wl_teacher` pattern in script 08, `load_als_teacher` in `pipeline/trainer_als.py`); this technique's only new code is the gate function and a training loop that calls both teachers per batch – structurally a merge of `trainer.py`'s online-KD loop and `trainer_als.py`'s paired-batch loop.

Why it fits this project

Directly resolves an open question this project already has: $WL \rightarrow WL$ distillation reaches median Dice 0.828; $ALS \rightarrow WL$ reaches 0.811. Rather than treating this as "which teacher wins," an ensemble asks whether the two signals are complementary enough that combining them beats either alone – plausible, since the ALS signal is fairness-relevant (Chapter on fairness distillation) in a way the WL teacher's own gap suggests it does not fully capture by itself.

Chapter 8

Deep Mutual Learning (Online Co-Distillation)

8.1 What it is

Every technique so far (and everything already in this project) needs a *frozen, pre-trained, larger* teacher. Deep Mutual Learning drops that requirement entirely: two students of *any* size train together from scratch, each treating the other’s live (not frozen) prediction as a soft target. Applied here: SegFormer-B0 and YOLO26n-sem train side by side, each nudging the other toward agreement, with no SegFormer-B2 teacher training step needed at all for this variant.

8.2 Architecture for this project

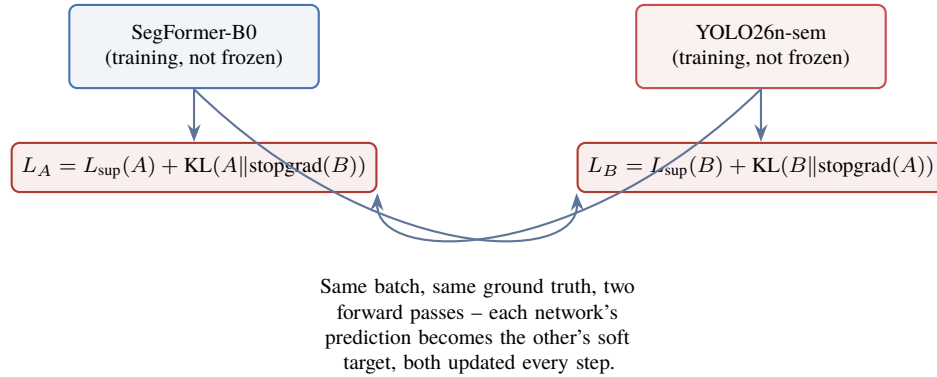


Figure 8.1: No pre-trained teacher anywhere in this diagram: both networks start from ImageNet/COCO pre-training only, and teach each other live, simultaneously, every training step.

8.3 Loss

$$L_A = L_{\text{sup}}(A) + \lambda \cdot \text{KL}(\sigma(z_A) \| \text{stopgrad}(\sigma(z_B))), \quad L_B = L_{\text{sup}}(B) + \lambda \cdot \text{KL}(\sigma(z_B) \| \text{stopgrad}(\sigma(z_A)))$$

Both networks are updated on their own loss every step; the `stopgrad` is essential – without it, this collapses into a single joint loss that does not actually implement mutual teaching.

Implementation hook

This does not fit neatly into either `trainer.py` (built around one frozen teacher closure) or the Ultralytics-managed YOLO training loop (script 06/08) – YOLO’s training is owned entirely by Ultralytics’ `model.train()`, which has no hook for injecting a second model’s live predictions into its loss. This would need a genuinely new training script that manually steps *both* optimizers together, more similar in spirit to `trainer_als.py`’s paired-batch loop than to either existing single-teacher path.

Caveat

The best-documented failure mode of mutual learning is both networks converging together toward a shared, comfortable-but-wrong local optimum early in training, before either has learned anything worth teaching. A short supervised-only warmup (several epochs of L_{sup} alone, $\lambda = 0$) before the mutual term switches on is standard practice and should not be skipped.

Chapter 9

Self-Distillation (Deep Supervision)

9.1 What it is

All prior techniques distill *between* two separate networks. Self-distillation distills a network *into itself*: attach small auxiliary segmentation heads to early/intermediate encoder stages, and train those shallow heads to match both the ground truth and the network’s own deepest, final-stage output (stop-gradient). The deep layers become a teacher for the shallow layers, inside one single model.

9.2 Architecture for this project

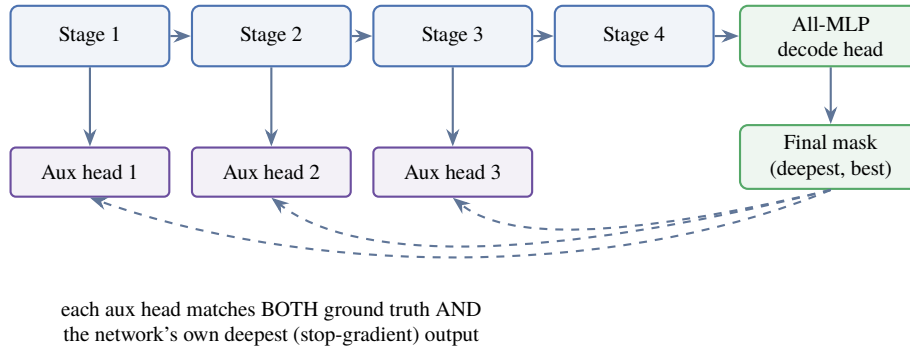


Figure 9.1: Three lightweight auxiliary heads, one per early/mid encoder stage, each trained against ground truth and against the final head’s own stop-gradient output – the network teaches its own shallow layers using its own deep layers, with no external model involved at all.

9.3 Loss

$$L = L_{\text{sup}}(\text{final}) + \sum_{i=1}^3 \left[\gamma_1 L_{\text{sup}}(\text{aux}_i) + \gamma_2 \text{KL}(\sigma(z_{\text{aux}_i}) \parallel \text{stopgrad}(\sigma(z_{\text{final}}))) \right]$$

Implementation hook

Applicable to *either* SegFormer network. Applied to the B2 teacher, it improves the teacher itself (a better teacher raises the ceiling for every other distillation technique in this document) at zero extra external-model cost. Applied to B0, the auxiliary heads become optional extra-tiny deployment candidates, extracted for

free alongside the main model.

Why it fits this project

Every other technique in this document assumes the teacher is already as good as it can be and focuses on transferring that fixed ceiling to a student. Self-distillation is the one technique here that *raises the ceiling itself* – worth considering as a first pass on the B2 teacher before any of the cross-model techniques, since it could shift every downstream number.

Chapter 10

Adversarial Fairness Distillation

10.1 What it is

Fair-KD Approach B (already implemented) pushes the student’s decode features toward matching the teacher’s per-ITA-group feature *moments* via MMD – effective, but MMD only checks whether two distributions’ means (in kernel space) agree, which can miss higher-order differences. Adversarial fairness distillation instead trains a small discriminator to explicitly try to *predict skin-tone group from the student’s features*, and trains the student (via a gradient-reversal layer) to defeat it – directly optimizing for the property actually wanted: a representation from which skin tone cannot be recovered at all.

10.2 Architecture for this project

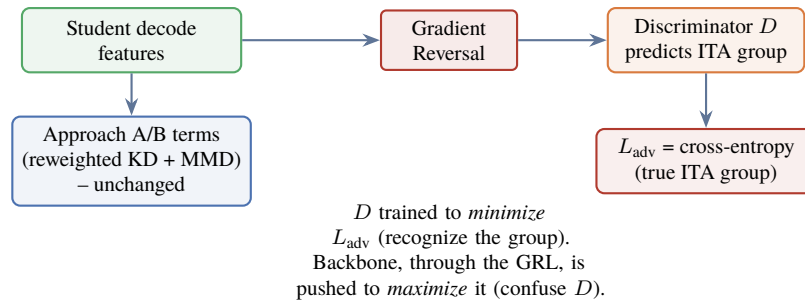


Figure 10.1: The Gradient Reversal Layer passes features through unchanged on the forward pass but flips the sign of the gradient on the backward pass – so the same loss trains the discriminator to succeed while training the student’s backbone to make it fail, both from one loss term.

10.3 Loss

$$L = \alpha L_{\text{sup}} + (1 - \alpha) L_{\text{KD, reweighted}} + \lambda_{\text{mmd}} L_{\text{MMD}} + \lambda_{\text{adv}} \text{CE}(D(\text{GRL}(F_s)), y_{\text{ITA}})$$

where the first three terms are exactly Approach B unchanged, and the last term is new: D is a small classifier (e.g. global-average-pool $\rightarrow$ 2-layer MLP $\rightarrow$ 5-way softmax over ITA groups), and GRL has no parameters of its own – it is the identity function forward, and negates the gradient (scaled by λ_{adv}) backward.

Implementation hook

Would extend `losses_fairness.py` with a new `Adversarial FairLoss` class alongside `FeatureAlignFairLoss`, reusing the exact same feature-extraction hook Approach B already has. The discriminator D is small enough (a few thousand parameters) that it adds negligible training cost.

Caveat

Adversarial training (GAN-style minimax) is well known to be less stable than a moment-matching loss like MMD – oscillation and mode collapse in the discriminator are real risks that MMD alone does not have. This is best framed as *Approach C*, run alongside (not instead of) Approach B for comparison, rather than assumed to be a strict upgrade.

Chapter 11

Compression-Aware Distillation

11.1 What it is

Every technique so far uses distillation to *train* a smaller architecture. This one uses distillation to *recover* accuracy lost during a separate compression step (structured pruning or INT8 quantization) applied *after* training already-finished, full-precision models – treating the compressed model as a new student and the already-distilled B0 (or even the B2 teacher) as its teacher.

11.2 Architecture for this project

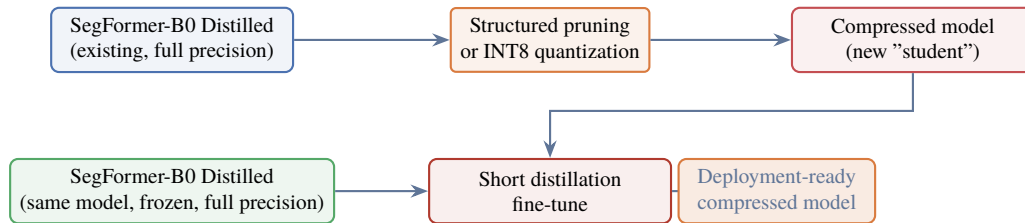


Figure 11.1: The already-distilled B0 model plays teacher one more time — for its own compressed version. Compression happens first (pruning/quantization, a mechanical transformation of the trained weights), then a short distillation fine-tune recovers whatever accuracy the compression step cost, using the exact same response-based KD loss already in the codebase.

11.3 Loss

Identical in form to the project’s existing `DistillSegLoss` — only the identity of teacher and student changes:

$$L = \alpha L_{\text{sup}} + (1 - \alpha) \text{BCE}(z_{\text{compressed}}, \sigma(z_{\text{B0-distilled}}/T))$$

Implementation hook

The loss function needs no new code at all – `pipeline/losses.py`’s existing `DistillSegLoss` is reused verbatim. The only new work is the compression step itself (a pruning or quantization library call) and a short additional training script that treats the already-trained B0 as a frozen teacher for its own compressed copy.

Why it fits this project

The project's own benchmark chapter shows SegFormer-B0 already reaching 86 FPS in the full pipeline – fast, but a further compression pass aimed at true embedded/mobile deployment (sub-10ms, INT8, on-device) is the natural next step if the deployment target ever moves off a GPU server, and distillation-guided compression consistently preserves more accuracy than compressing without it.

Chapter 12

Prioritized Recommendation

Given a limited GPU budget (the project is currently mid-way through a single extended training session), not all nine techniques are equally worth trying first. This chapter ranks them by a simple heuristic: *implementation cost* $\times$ *risk* vs. *expected payoff*, using only what already exists in the codebase as a starting point.

#	Technique	New params	New training scripts	Reuses existing code
1	Boundary-Aware	none	no (edit existing loss)	distance-transform from <code>metrics_extended.py</code>
2	Attention Transfer	none	no (edit existing loss)	hidden states already returned by HF SegFormer
3	Feature (Hint)	small (adapters)	no (edit existing loss)	same hidden-state hook as #2
4	Structured/Channel-wise	none–small	no (edit existing loss)	Approach B’s feature hook
5	Multi-Teacher Ensemble	none–small (gate)	yes, but merges two existing loops	both teacher-loading paths exist
6	Compression-Aware	none	yes, short fine-tune only	<code>DistillSegLoss</code> verbatim
7	Adversarial Fairness	small (discriminator)	no (edit existing loss)	Approach B’s feature hook
8	Self-Distillation	small (aux heads)	yes, new training loop	none, genuinely new
9	Deep Mutual Learning	none (2 existing models)	yes, genuinely new dual-optimizer loop	none, genuinely new

Recommended order of experimentation

1. **Boundary-Aware Distillation** first — it is a same-day change (reweight an existing loss term with code that already exists for a different purpose), directly targets a metric this project already tracks and cares about (Surface Dice/HD95), and cannot make anything worse than the current loss if σ is set conservatively.
2. **Attention Transfer**, run alongside #1 — equally cheap, zero new parameters, complementary signal (spatial focus rather than boundary sharpness).
3. **Multi-Teacher Ensemble (WL+ALS)** once the current `als_to_wl_distill` run finishes — it di-

rectly answers a question this project already has data for (does combining the two existing teachers beat either alone), and both halves of the implementation already exist independently.

4. Everything else, in order, as GPU budget allows: Feature-Hint and Structured/Channel-wise (moderate cost, incremental over #1–#2), then Adversarial Fairness (worth comparing against Approach B directly once corrected YOLO fairness numbers exist), then Compression-Aware (only once a deployment target more constrained than the current GPU server is actually on the roadmap), and Self-Distillation / Deep Mutual Learning last, since both require genuinely new training infrastructure rather than incremental edits.

Appendix: Formula Reference

Feature/Hint:	$L_{\text{feat}} = \sum_i \ A_i(f_i^s) - \text{stopgrad}(f_i^t)\ _2^2$
Attention Transfer:	$A_i = \text{norm}(\text{vec}(\sum_c F_i[c] ^2)), \quad L_{AT} = \sum_i \ A_i^t - A_i^s\ _2$
Channel-wise:	$L_{CW} = \sum_c \text{KL}(\sigma(F^t[c]/T) \parallel \sigma(F^s[c]/T))$
Relational/pairwise:	$L_{rel} = \ \text{sim}(F^t) - \text{sim}(F^s)\ _2^2$
Boundary-aware:	$w(x, y) = \exp(-d(x, y)^2/2\sigma^2), \quad L_{KD}^{\text{bd}} = \text{mean}[w \cdot \text{BCE}]$
Multi-teacher ensemble:	$p_{\text{ens}} = g \cdot p_{WL} + (1 - g) \cdot p_{ALS \rightarrow WL}$
Deep Mutual Learning:	$L_A = L_{\text{sup}}(A) + \lambda \text{KL}(\sigma(z_A) \parallel \text{stopgrad}(\sigma(z_B)))$
Self-distillation:	$L = L_{\text{sup}}(\text{final}) + \sum_i [\gamma_1 L_{\text{sup}}(\text{aux}_i) + \gamma_2 \text{KL}(\cdot \parallel \text{stopgrad}(\cdot))]$
Adversarial fairness:	$L = \alpha L_{\text{sup}} + (1 - \alpha) L_{KD} + \lambda_{mmd} L_{MMD} + \lambda_{adv} \text{CE}(D(\text{GRL}(F_s)), y_{ITA})$
Compression-aware:	$L = \alpha L_{\text{sup}} + (1 - \alpha) \text{BCE}(z_{\text{compressed}}, \sigma(z_{B0}/T))$